UNIT 02

DATA PREPROCESSING, ANALYSIS AND VISUALIZATION

STUDENT NOTES

Syllabus coverage:

Mean Removal • Scaling • Normalization • Binarization • One-Hot Encoding • Label Encoding • Loading and Summarizing Data • Univariate and Multivariate Plots • Training and Test Data • Performance Measures

Understand the data → Prepare it correctly → Visualize it → Evaluate the model

A. How to Use These Notes

- **Start with the basics:** learn sample, feature, target, numeric data and categorical data before studying transformations.
- **Work through every calculation:** the small numerical examples explain what the library functions do internally.
- **Run the Python code:** type the programs yourself and compare the input with the transformed output.
- **Revise with comparisons:** pay special attention to scaling versus normalization and label versus one-hot encoding.
- **Test yourself:** attempt the MCQs, short questions and practical exercises at the end.

STUDY RULE	For every preprocessing technique, remember three things: why it is needed, what it changes, and when it should be used.

B. Learning Outcomes

- Explain why raw data usually requires preprocessing.
- Perform mean removal, standardization, min-max scaling, normalization and binarization.
- Convert categorical data using label encoding and one-hot encoding.
- Load and summarize a dataset with pandas.
- Choose suitable univariate and multivariate plots.
- Differentiate training data and test data and avoid data leakage.
- Calculate and interpret common classification and regression measures.

C. Contents

1. Basic Dataset Concepts

1.1 What is a Dataset?

A dataset is an organized collection of related observations. In a table, each row usually represents one observation and each column represents one characteristic of that observation.

Student_ID	Hours_Studied	Attendance	Department	Result
S01	2	68	CSE	Fail
S02	5	82	IT	Pass
S03	7	91	CSE	Pass
S04	3	74	ECE	Fail

Term	Meaning	Example from table
Sample / observation	One row of the dataset	The complete record of student S02
Feature	An input variable used to describe or predict	Hours_Studied, Attendance, Department
Target / label	The output that a model tries to predict	Result
Numeric feature	A feature represented by numbers	Attendance = 82
Categorical feature	A feature containing categories or names	Department = IT

MEMORY AID	Rows are samples. Columns are variables. Input columns are features. The output column is the target.

1.2 Why Data Needs Preparation

- Features may use very different ranges, such as age in years and income in thousands.
- Machine-learning algorithms generally require numbers, but datasets may contain text categories.
- Values may be missing, duplicated, inconsistent or incorrectly typed.
- A model may learn misleading patterns if test data influences preprocessing.
- Charts and summaries are needed to understand distributions, relationships and unusual values.

2. Data Preprocessing Overview

2.1 Meaning

Data preprocessing is the set of operations used to convert raw data into a clean and suitable form for analysis or machine learning. It improves consistency, comparability and model reliability.

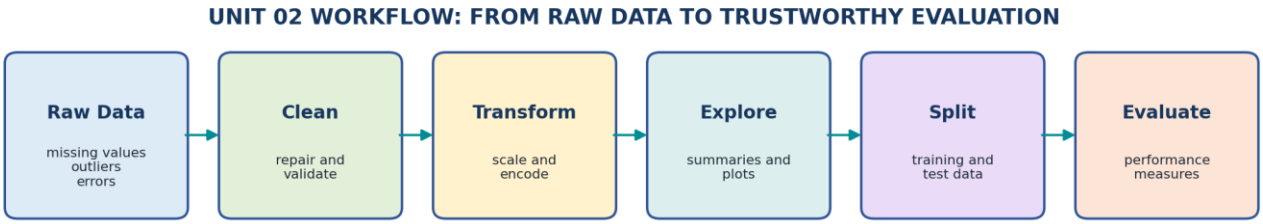

Figure 1: A simple workflow from raw data to model evaluation.

Problem in data	Possible preprocessing operation
Features have different scales	Standardization or min-max scaling
Rows have different vector lengths	Normalization
Only yes/no information is required	Binarization
Categories are written as text	Label, ordinal or one-hot encoding
Values are missing or duplicated	Cleaning and validation
Dataset is too large or complex	Selection, sampling or reduction

KEY POINT	Preprocessing does not mean changing data randomly. Every transformation must be selected according to the data type, algorithm and problem objective.

3. Mean Removal and Scaling

3.1 Mean Removal (Centering)

Mean removal subtracts the mean of a feature from every value. After centering, the transformed feature has a mean of zero.

FORMULA	Centered value = Original value - Mean of the feature

Example: values = [10, 20, 30, 40]. Mean = (10 + 20 + 30 + 40) / 4 = 25.

Original value	Calculation	Centered value
10	10 - 25	-15
20	20 - 25	-5
30	30 - 25	5
40	40 - 25	15

KEY POINT	Mean removal changes the centre of the feature but does not automatically make its standard deviation equal to 1.

3.2 Standardization

Standardization first removes the mean and then divides by the standard deviation. The transformed feature normally has mean 0 and standard deviation 1.

FORMULA	z = (x - mean) / standard deviation

- **Useful for:** algorithms affected by feature scale, such as k-nearest neighbours, support vector machines, logistic regression and many gradient-based methods.
- **Result:** values are not restricted to a fixed range; negative and positive values are common.
- **Outliers:** because mean and standard deviation are affected by extreme values, standardization can also be influenced by outliers.

Python: mean removal and standardization

```
import numpy as np
from sklearn.preprocessing import StandardScaler

X = np.array([[10], [20], [30], [40]])
scaler = StandardScaler()
X_standard = scaler.fit_transform(X)

print(scaler.mean_)
print(X_standard)
```

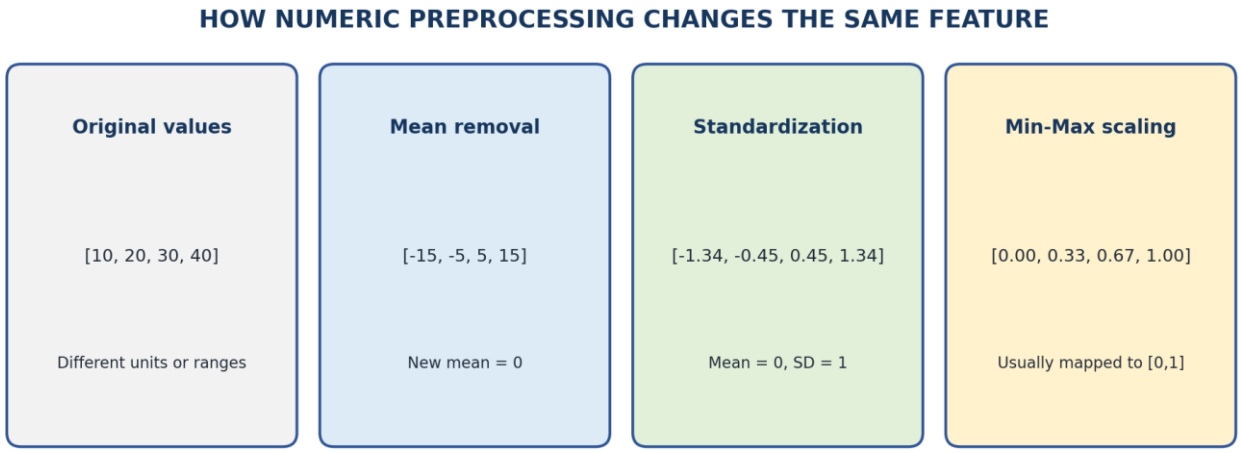

Figure 2: Comparison of centering, standardization and min-max scaling.

3.3 Min-Max Scaling

Min-max scaling maps each feature into a selected range, commonly 0 to 1. The smallest training value becomes 0 and the largest becomes 1.

FORMULA	x_scaled = (x - minimum) / (maximum - minimum)

For [10, 20, 30, 40], minimum = 10 and maximum = 40. The scaled value of 20 is (20 - 10) / (40 - 10) = 0.33.

Python: min-max scaling

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

X = np.array([[10], [20], [30], [40]])
scaler = MinMaxScaler(feature_range=(0, 1))
X_scaled = scaler.fit_transform(X)
print(X_scaled)
```

Method	Main transformation	Usual result	Important point
Mean removal	Subtract feature mean	Mean becomes 0	Scale is unchanged
Standardization	Subtract mean and divide by SD	Mean 0, SD 1	No fixed minimum/maximum
Min-max scaling	Use feature minimum and maximum	Usually values in [0,1]	Extreme values determine the range

EXAM TIP	Scaling is normally performed column by column because each column represents a different feature.

4. Normalization and Binarization

4.1 Normalization

Normalization rescales each sample so that the complete row has unit length. It is different from feature scaling because it works row by row rather than column by column.

FORMULA	L2 norm of [a, b] = sqrt(a^2 + b^2). Normalized row = row / its L2 norm.

Example: row = [3, 4]. L2 norm = sqrt(3^2 + 4^2) = 5. Normalized row = [3/5, 4/5] = [0.6, 0.8].

Original row	L2 norm	Normalized row
[3, 4]	5	[0.6, 0.8]
[5, 12]	13	[0.385, 0.923]

Python: row normalization

```
import numpy as np
from sklearn.preprocessing import Normalizer

X = np.array([[3, 4], [5, 12]])
normalizer = Normalizer(norm='l2')
X_normal = normalizer.transform(X)
print(X_normal)
```

WHEN USEFUL	Normalization is often useful when direction or relative proportions matter more than total magnitude, such as some text-vector and similarity problems.

Question	Scaling	Normalization
Works on	Each feature / column	Each sample / row
Main purpose	Make features comparable	Make sample length equal to 1
Uses information from other rows?	Yes, feature statistics are learned from training data	No, each row is transformed using its own values
Typical tool	StandardScaler or MinMaxScaler	Normalizer

4.2 Binarization

Binarization converts numeric values into two values, usually 0 and 1, according to a threshold. Values greater than the threshold become 1; other values become 0.

RULE	Output = 1 when value > threshold; otherwise output = 0.

Example: marks = [35, 48, 62, 80] and threshold = 50. Output = [0, 0, 1, 1].

Python: binarization

```
import numpy as np
from sklearn.preprocessing import Binarizer

X = np.array([[35], [48], [62], [80]])
binarizer = Binarizer(threshold=50)
print(binarizer.transform(X))
```

CAUTION	Binarization removes information. After conversion, 62 and 80 both become 1, so their original difference is lost.

5. Encoding Categorical Data

5.1 Why Encoding is Needed

Many machine-learning algorithms operate on numbers. Therefore, text categories such as department, city or colour must be represented numerically without creating false meaning.

CATEGORICAL ENCODING EXAMPLE

City	Label code	Kanpur	Delhi	Lucknow
Kanpur	0	1	0	0
Delhi	1	0	1	0
Lucknow	2	0	0	1

Label encoding: one integer per category

One-hot encoding: one binary column per category

Figure 3: Label codes and one-hot columns for the same city categories.

5.2 Label Encoding

Label encoding assigns one integer to each class. It is mainly appropriate for the target variable in a classification problem.

Original target	Encoded target
Fail	0
Pass	1
Excellent	2

Python: label encoding a target

```
from sklearn.preprocessing import LabelEncoder

y = ['Fail', 'Pass', 'Pass', 'Excellent']
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

print(encoder.classes_)
print(y_encoded)
```

CAUTION	Do not automatically use LabelEncoder for an unordered input feature such as City. Codes 0, 1 and 2 can incorrectly suggest order or distance.

5.3 One-Hot Encoding

One-hot encoding creates one binary column for every category. Each sample receives 1 in the column representing its category and 0 in the other category columns.

City	City_Delhi	City_Kanpur	City_Lucknow
Kanpur	0	1	0
Delhi	1	0	0
Lucknow	0	0	1

Python: one-hot encoding an input feature

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder

X = np.array([['Kanpur'], ['Delhi'], ['Lucknow']])
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
X_encoded = encoder.fit_transform(X)

print(encoder.get_feature_names_out())
print(X_encoded)
```

Point	Label encoding	One-hot encoding
Output	One integer column	One binary column per category

Point	Label encoding	One-hot encoding
Best basic use	Target labels	Unordered input categories
Possible problem	May create false order for input data	Can create many columns when categories are numerous
Example	Pass -> 1	City_Kanpur -> 1

MEMORY AID	LabelEncoder is generally for y (target). OneHotEncoder is generally for categorical columns in X (features).

6. Loading and Summarizing a Dataset

6.1 Loading a CSV File

pandas represents tabular data using a DataFrame. The read_csv() function loads a comma-separated values file into a DataFrame.

Python: load a dataset

```
import pandas as pd

df = pd.read_csv('students.csv')
print(df.head())
```

Command	Purpose
df.head()	Display the first 5 rows
df.tail()	Display the last 5 rows
df.shape	Return (number of rows, number of columns)
df.columns	Display column names
df.dtypes	Display the data type of each column
df.info()	Print a compact summary of columns, non-null counts and data types
df.describe()	Calculate numeric summary statistics

OUTPUT READING	If df.shape returns (150, 5), the dataset contains 150 rows and 5 columns.

6.2 Numerical Summary

Statistic	Meaning
count	Number of non-missing values
mean	Arithmetic average
std	Standard deviation; indicates spread

Statistic	Meaning
min	Smallest value
25%	First quartile
50%	Median
75%	Third quartile
max	Largest value

Python: common summaries and quality checks

```
print(df.describe())
print(df['Department'].value_counts())
print(df.isnull().sum())
print(df.duplicated().sum())
```

Question about data	Useful command
How many rows and columns?	df.shape
What are the column names?	df.columns
Which columns are numeric or text?	df.dtypes or df.info()
What is the average and spread?	df.describe()
How many records belong to each category?	df[column].value_counts()
How many values are missing?	df.isnull().sum()
How many duplicate rows exist?	df.duplicated().sum()

STUDY HABIT	Never start modelling immediately after loading a file. First inspect rows, shape, data types, missing values, duplicates and summary statistics.

7. Univariate Data Visualization

7.1 Meaning

A univariate plot studies one variable at a time. It helps us understand frequency, centre, spread, skewness and possible outliers.

Data type / purpose	Suitable plot	What it shows
One categorical variable	Bar chart / count plot	Frequency of each category
One numeric variable	Histogram	Shape and frequency distribution
One numeric variable	Box plot	Median, quartiles, spread and possible outliers
Ordered values over time	Line plot	Trend across time or sequence

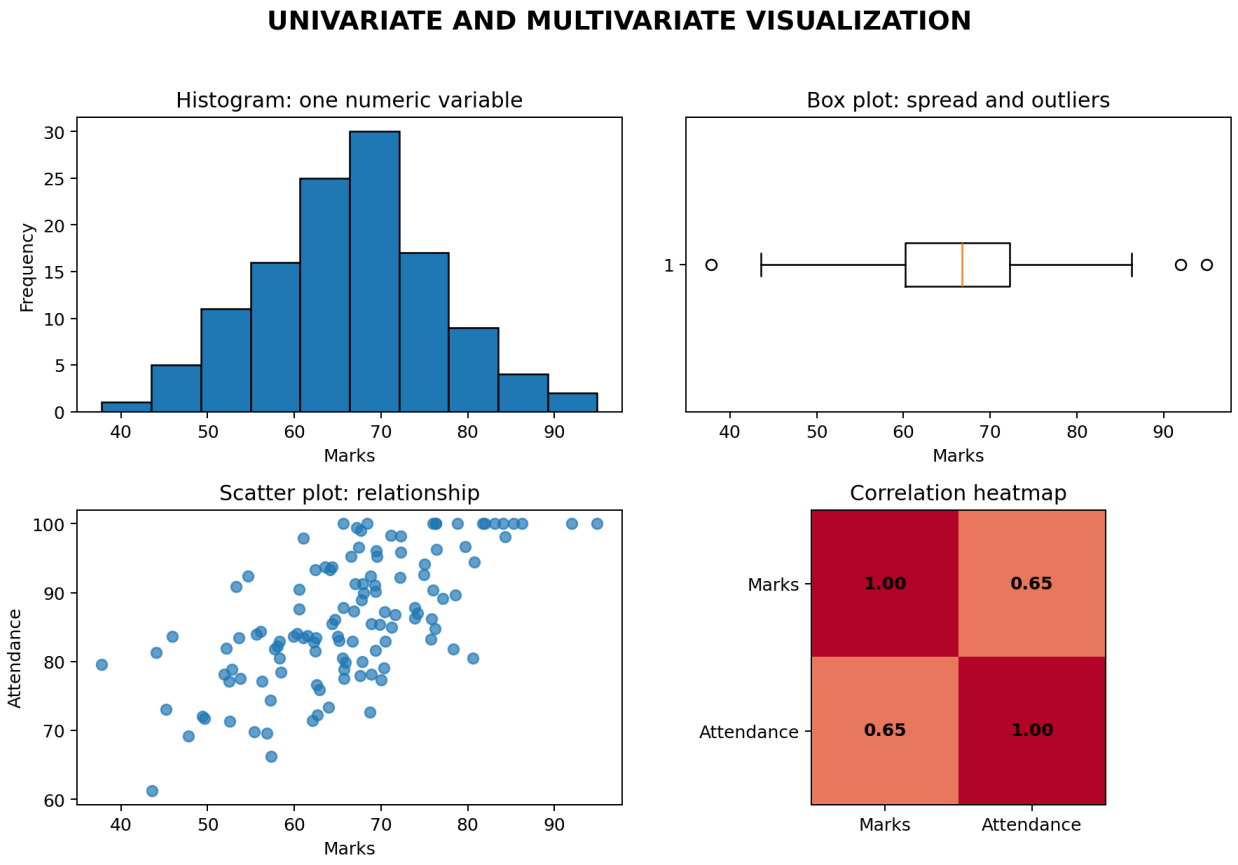

Figure 4: Examples of histogram, box plot, scatter plot and correlation heatmap.

7.2 Histogram

A histogram divides numeric values into intervals called bins and shows how many observations fall in each interval. It is used for continuous or numeric data, not category names.

Python: histogram

```
import matplotlib.pyplot as plt

df['Marks'].plot(kind='hist', bins=8, edgecolor='black')
plt.xlabel('Marks')
plt.title('Distribution of Marks')
plt.show()
```

7.3 Bar Chart

Python: bar chart for category counts

```
df['Department'].value_counts().plot(kind='bar')
plt.xlabel('Department')
plt.ylabel('Number of students')
plt.title('Students by Department')
plt.show()
```

7.4 Box Plot

A box plot summarizes a numeric distribution through quartiles. Points far from the main range may be shown as possible outliers, but they should be investigated rather than automatically deleted.

Python: box plot

```
df['Marks'].plot(kind='box')
plt.title('Box Plot of Marks')
plt.show()
```

EXAM TIP	Bar charts compare categories. Histograms show the distribution of numerical values grouped into bins.

8. Multivariate Data Visualization

8.1 Meaning

A multivariate plot studies two or more variables together. It is used to identify relationships, group differences, interactions and patterns that are not visible in one variable alone.

Plot	Variables compared	Main use
Scatter plot	Two numeric variables	Study direction, strength and pattern of association
Grouped / stacked bar chart	Two categorical variables or category plus aggregate	Compare subgroups
Box plot by category	One numeric and one categorical variable	Compare distributions across groups
Pair plot	Several numeric variables	View many pairwise relationships
Correlation heatmap	Several numeric variables	Display a correlation matrix

8.2 Scatter Plot

Python: scatter plot

```
import matplotlib.pyplot as plt

plt.scatter(df['Hours_Studied'], df['Marks'])
plt.xlabel('Hours Studied')
plt.ylabel('Marks')
plt.title('Hours Studied vs Marks')
plt.show()
```

8.3 Correlation Heatmap

Correlation measures the direction and strength of a linear relationship between numeric variables. Its value ranges from -1 to +1.

Correlation value	Basic interpretation
Near +1	Strong positive linear relationship
Near 0	Weak or no linear relationship
Near -1	Strong negative linear relationship

Python: simple correlation heatmap

```
import matplotlib.pyplot as plt

corr = df.select_dtypes(include='number').corr()
plt.imshow(corr, vmin=-1, vmax=1, cmap='coolwarm')
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

CAUTION	Correlation does not prove causation. Two variables can move together because of coincidence, a third variable or a complex relationship.

9. Training Data and Test Data

9.1 Why Split a Dataset?

A model must be evaluated on data that it did not use for learning. Therefore, the available dataset is commonly divided into training data and test data.

Part	Purpose	Typical proportion
Training data	Used to learn model parameters and fit preprocessing operations	About 70% to 80%
Test data	Used for final evaluation on unseen examples	About 20% to 30%
Validation data (when used)	Used for model selection and tuning before final testing	Taken separately or created through cross-validation

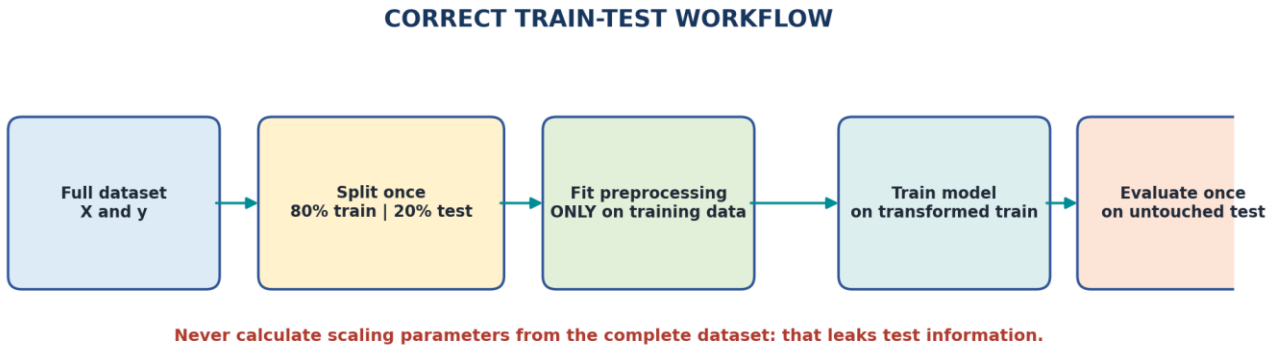

Figure 5: Correct order for splitting, preprocessing, training and evaluation.

Python: train-test split

```
from sklearn.model_selection import train_test_split

X = df[['Hours_Studied', 'Attendance']]
y = df['Result']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print(X_train.shape)
print(X_test.shape)
```

Argument	Meaning
test_size=0.20	20% of records are placed in the test set
random_state=42	Makes the random split reproducible
stratify=y	Keeps class proportions similar in training and test sets

9.2 Data Leakage

Data leakage occurs when information that should be unavailable during training enters the learning process. Leakage produces unrealistically good evaluation results and poor performance on truly new data.

RULE	Split first. Fit scalers and encoders only on training data. Use the fitted objects to transform the test data.

Correct fitting of a scaler

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)  # no fit on test data
```

10. Performance Measures

10.1 Classification and Regression

Problem type	Output	Example
Classification	A category or class	Pass / Fail, Spam / Not Spam
Regression	A continuous numeric value	Marks, price, temperature

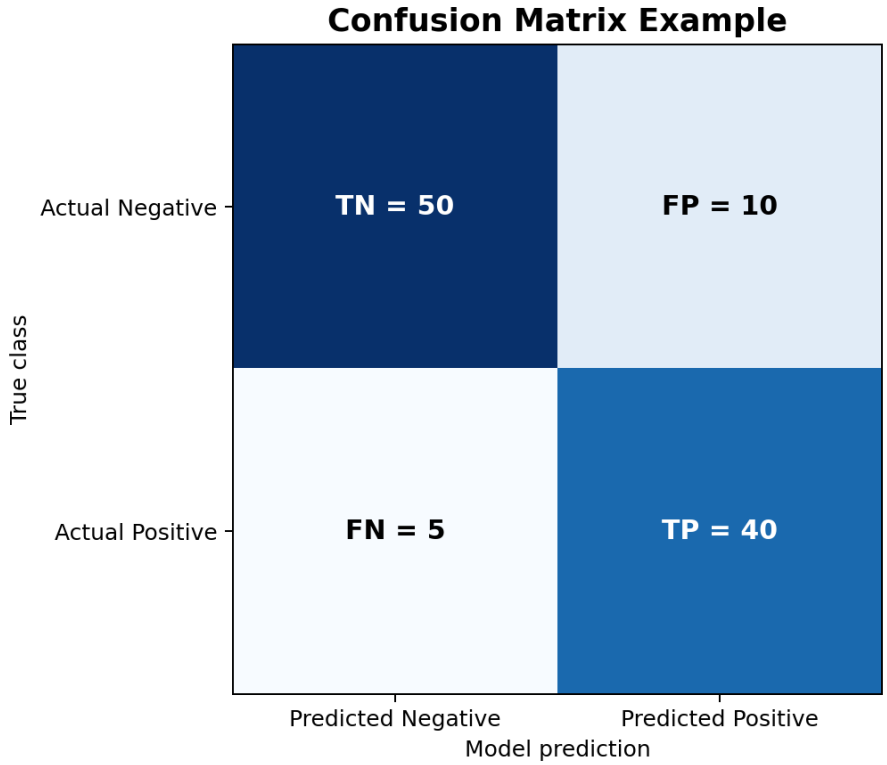

Figure 6: Confusion matrix terms for binary classification.

Term	Meaning
True Positive (TP)	Positive case correctly predicted as positive
True Negative (TN)	Negative case correctly predicted as negative
False Positive (FP)	Negative case incorrectly predicted as positive
False Negative (FN)	Positive case incorrectly predicted as negative

10.2 Classification Measures

Measure	Formula	Meaning
Accuracy	(TP + TN) / Total	Overall fraction of correct predictions
Precision	TP / (TP + FP)	Of predicted positives, how many were correct?
Recall	TP / (TP + FN)	Of actual positives, how many were found?
F1-score	2 x Precision x Recall / (Precision + Recall)	Balance between precision and recall

Using the confusion matrix in Figure 6: TP = 40, TN = 50, FP = 10 and FN = 5.

Measure	Calculation	Result
Accuracy	(40 + 50) / 105	0.857 or 85.7%
Precision	40 / (40 + 10)	0.800 or 80.0%
Recall	40 / (40 + 5)	0.889 or 88.9%

Measure	Calculation	Result
F1-score	2 x 0.800 x 0.889 / (0.800 + 0.889)	About 0.842

CHOOSING A METRIC	Accuracy can be misleading when one class is much more common. Use precision, recall, F1-score and the confusion matrix to understand the types of errors.

Python: classification measures

```
from sklearn.metrics import accuracy_score, precision_score
from sklearn.metrics import recall_score, f1_score, confusion_matrix

print(accuracy_score(y_test, y_pred))
print(precision_score(y_test, y_pred, pos_label='Pass'))
print(recall_score(y_test, y_pred, pos_label='Pass'))
print(f1_score(y_test, y_pred, pos_label='Pass'))
print(confusion_matrix(y_test, y_pred))
```

10.3 Regression Measures

Measure	Meaning	Better value
MAE: Mean Absolute Error	Average absolute difference between actual and predicted values	Smaller
MSE: Mean Squared Error	Average squared error; gives more weight to large errors	Smaller
RMSE: Root Mean Squared Error	Square root of MSE; same unit as the target	Smaller
R-squared (R2)	Proportion of target variation explained relative to a mean baseline	Closer to 1, interpreted with context

FORMULAS	MAE = average(|actual - predicted|); MSE = average((actual - predicted)^2); RMSE = sqrt(MSE).

Python: regression measures

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)

print(mae, mse, rmse, r2)
```

11. Complete Practical Workflow

11.1 Iris Classification Example

The following program loads the Iris dataset, summarizes it, creates a training-test split, standardizes the numeric features, trains a logistic-regression classifier and evaluates accuracy. A Pipeline keeps preprocessing and modelling in the correct order.

Complete scikit-learn practical

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Load data
iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

# 2. Summarize data
print(X.head())
print(X.shape)
print(X.describe())
print(y.value_counts())

# 3. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# 4. Build preprocessing + model pipeline
model = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(max_iter=1000))
])

# 5. Train and predict
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# 6. Evaluate
print('Accuracy:', accuracy_score(y_test, y_pred))
print('Confusion matrix:
', confusion_matrix(y_test, y_pred))
```

1. load_iris(as_frame=True) provides the feature table and target in pandas form.
2. train_test_split creates unseen test examples.
3. StandardScaler learns mean and standard deviation only from training data because it is inside the pipeline.
4. LogisticRegression learns class boundaries from the transformed training data.
5. accuracy_score and confusion_matrix evaluate predictions on the test set.

PRACTICE EXTENSION	Replace StandardScaler with MinMaxScaler. Run the program again and compare the test result. Then draw a histogram for each Iris feature.

11.2 Recommended Order for Any Small Project

1. Define the target and identify features.
2. Load the dataset and inspect its first rows.
3. Check shape, data types, missing values, duplicates and class counts.
4. Create suitable univariate and multivariate plots.
5. Split the dataset into training and test parts.
6. Fit preprocessing operations on training data only.
7. Train the selected model.
8. Predict test data and calculate suitable performance measures.
9. Interpret errors and improve the workflow without using the test set repeatedly.

12. Quick Revision and Glossary

12.1 One-Page Revision

Topic	Remember this
Mean removal	Subtract column mean; new mean becomes 0
Standardization	Column-wise; mean 0 and standard deviation 1
Min-max scaling	Column-wise; commonly maps values to 0 to 1
Normalization	Row-wise; sample vector receives unit length
Binarization	Convert values to 0/1 using a threshold
Label encoding	Convert target classes to integer labels
One-hot encoding	Create one binary feature for every input category
Univariate plot	Study one variable
Multivariate plot	Study relationships among two or more variables
Training data	Used to learn preprocessing and model parameters
Test data	Held back for final evaluation
Accuracy	Overall correctness
Precision	Correctness of positive predictions
Recall	Coverage of actual positive cases
F1-score	Balance of precision and recall
MAE / RMSE	Regression error measures in target units

MASTER MEMORY AID	Columns are scaled. Rows are normalized. Categories are encoded. Training data teaches. Test data checks.

12.2 Glossary

Term	Simple meaning
Bin	An interval used in a histogram
Category	A named group such as CSE, IT or ECE
Data leakage	Unfair use of unavailable test information during training
Distribution	How values are spread and how frequently they occur
Fit	Learn parameters from data
Outlier	A value far from the main pattern
Pipeline	An ordered sequence of preprocessing and modelling steps
Random state	A fixed seed that makes random results reproducible

Term	Simple meaning
Stratification	Preserving class proportions during splitting
Transform	Apply a learned or defined data conversion

13. Self-Assessment

13.1 Multiple-Choice Questions

1. Which operation subtracts the feature mean from every value?

A. Binarization
B. Mean removal
C. One-hot encoding
D. Testing

2. Standardization normally produces a feature with:

A. Mean 0 and SD 1
B. Values only 0 and 1
C. One column per category
D. No negative values

3. Which technique usually works row by row?

A. Min-max scaling
B. Standardization
C. Normalization
D. Label encoding

4. Which encoder is generally used for an unordered input category?

A. OneHotEncoder
B. StandardScaler
C. Binarizer
D. Normalizer

5. Which pandas command reports rows and columns?

A. df.shape
B. df.mean_only()
C. df.plot_all()
D. df.labels

6. A histogram is mainly used for:

A. One numeric variable
B. Only text categories
C. Model training
D. Encoding

7. Which plot studies two numeric variables together?

A. Scatter plot
B. Pie chart only
C. One-hot table
D. Confusion matrix only

8. The test set should be used to:

A. Learn scaler statistics
B. Fit the model repeatedly
C. Evaluate unseen-data performance
D. Create the target

9. Which measure answers: Of actual positives, how many were found?

A. Precision
B. Recall
C. MAE
D. R2

10. Which is a regression measure?

A. RMSE
B. Recall
C. Precision
D. Confusion matrix

13.2 Short-Answer Questions

1. Define data preprocessing and state two reasons for using it.
2. Differentiate mean removal and standardization.
3. Differentiate scaling and normalization.
4. Explain binarization with a numerical example.
5. Why can label encoding be unsuitable for an unordered input feature?
6. Write any five pandas commands used for dataset summarization.
7. Differentiate a histogram and a bar chart.
8. What is the purpose of training and test data?
9. Define data leakage.
10. Differentiate precision and recall.

13.3 Long-Answer Questions

1. Explain mean removal, standardization and min-max scaling with formulas, examples and Python syntax.
2. Explain normalization, binarization, label encoding and one-hot encoding. State the correct use of each.
3. Describe the steps used to load and summarize a dataset using pandas.
4. Explain univariate and multivariate plots with suitable examples.
5. Explain the correct training-test workflow and the problem of data leakage.
6. Explain a confusion matrix and calculate accuracy, precision, recall and F1-score.
7. Explain MAE, MSE, RMSE and R-squared for regression evaluation.

13.4 Practical Exercises

1. Create a NumPy array [15, 25, 35, 45] and apply StandardScaler and MinMaxScaler.
2. Normalize the rows [3, 4] and [8, 15] using L2 normalization.
3. Binarize marks [32, 49, 50, 63, 91] using threshold 50 and explain the result.
4. One-hot encode the departments CSE, IT, ECE and CSE.
5. Load any CSV file and print head, shape, data types, describe, missing counts and duplicate counts.
6. Create a histogram, bar chart, box plot and scatter plot from a small student dataset.
7. Split a classification dataset into 75% training and 25% test data using stratification.
8. Train a simple classifier and report accuracy and confusion matrix.

13.5 MCQ Answer Key

Question	Answer	Reason in one line
1	B	Mean removal subtracts the feature mean.
2	A	Standardization centres and scales to unit standard deviation.

Question	Answer	Reason in one line
3	C	Normalization usually transforms each sample independently.
4	A	One-hot encoding avoids false order among nominal categories.
5	A	shape returns the row-column dimensions.
6	A	A histogram displays a numeric distribution.
7	A	A scatter plot compares two numeric variables.
8	C	The test set estimates performance on unseen data.
9	B	Recall measures the fraction of actual positives found.
10	A	RMSE is a regression error measure.

FINAL CHECK	Can you explain why scaling is column-wise, normalization is row-wise, and preprocessing must be fitted only on training data? If yes, the central ideas of Unit 02 are clear.

END OF UNIT 02

Revise concepts, run the programs, and interpret every output.